# Cooking Companion

*Solution architecture — a hands-free cooking assistant you talk to while your hands are busy.*

Next.js 16 · React 19 · TypeScript (strict)
Supabase Postgres + Auth · Vercel
Groq (LLM + Whisper) · Rime (TTS)

## THE PROBLEM, AND THE CONSTRAINT THAT SHAPES EVERYTHING

A cook has wet hands, a pan on the heat, and no attention to spare for a screen. So the product is **conversation, not documents**: you open it, say what you want to make, and it plans the recipe, walks you through it, tracks what is already on the stove, and reminds you before the pasta boils over. Nothing is uploaded and nothing is read.

That constraint decides the architecture. If the assistant answers a sentence nobody said, talks over a correction, or takes three seconds to react, the product has failed in a way no feature can compensate for. **Latency, interruption and false triggers are treated as correctness problems** — each has tests, not just tuning.

## THE TURN PIPELINE

**CAPTURE**
Mic → AudioWorklet. A pure VAD state machine reads RMS windows; a 400 ms pre-roll ring buffer keeps the first word the detector needed to become confident.

**ENDPOINT**
A pause at 420 ms starts transcription *inside* the 1100 ms hangover. If speech resumes, the work is discarded; if not, it was the whole utterance.

**UNDERSTAND**
Whisper → silence-artefact filter. Groq model streams the first pass, choosing among 11 tools that read and write the user's own data.

**SPEAK**
Rime synthesises each clause as it completes. One SSE stream carries events *and* PCM audio; a jitter-buffered worklet plays it.

**Audio rides the same request as the events.** That is the load-bearing decision: a barge-in becomes a single `fetch` abort, which propagates to Rime, the model and any in-flight tool at once. Split across two requests, one of them survives to talk over the user's correction.

## INTERRUPTION, AND KNOWING WHAT WAS HEARD

The player reports samples rendered per speech context; Rime returns word timestamps. Multiplying the two gives the **heard-transcript ledger** — what actually reached the room before the cook cut in.

That ledger is written over the interrupted turn *before* the next turn's history is read, so the model never treats a sentence nobody heard as context for the correction it caused. The interrupted path is the one place the pipeline stays deliberately sequential.

## ISOLATION AS A PROPERTY, NOT A RULE

Identity enters the system in exactly one place, from the verified session cookie. No route, tool or query accepts a user id from the outside, so "never trust a `user_id` from the client" is structural rather than remembered.

Postgres RLS is **enabled and forced** on every user table (8 policies), the server client carries the user's own JWT so the database applies the same rules, and every read *also* filters on `user_id` — three independent layers, each tested.

## LATENCY: MEASURED, NOT ASSERTED

| WHERE THE WAIT WAS | BEFORE | AFTER |
|---|---|---|
| Database work before the model is called | `~900 ms` | `~300 ms` |
| Session verification per API call | `300 ms` | `152 ms` |
| Request → first audio (median) | `2.9–3.4 s` | `2.76 s` |
| Hangover hidden behind transcription | `0 ms` | `680 ms` |

The prologue to a turn was six database round trips in a chain; only one dependency was real. They now run as one wave, and the user's line is written without being awaited. The first model pass is streamed and spoken clause by clause instead of awaited whole.

A test double can be given an artificial round-trip cost, so **re-chaining those queries fails the suite** rather than quietly costing half a second. Every figure above was measured against live services and verified failing on the previous code first.

## FAILURE IS DESIGNED FOR

- **Rate limits:** a short limit is waited out; a long one is answered on a second model, metered separately. Both exhausted → a spoken sentence a cook can act on, never a status code.
- **Silence artefacts:** transcribers return "Thank you." for noise. Energy gating, a duplicate-model check and a self-echo filter cut every link in that feedback loop.
- **Errors are spoken,** not just displayed. A hands-free product that fails silently looks identical to one that ignored you.

## SHAPE OF THE CODEBASE

- **84 TypeScript modules**, strict mode with `noUncheckedIndexedAccess`; no file does two jobs.
- **7 runtime dependencies.** The WAV encoder, jitter buffer and VAD are ours because a package would have cost more than it saved.
- The turn is an **async generator**, so the transport is not baked in: the route serialises it as SSE and tests consume the identical generator with fake providers.
- Provider-specific code sits behind one seam each — swapping a TTS or LLM vendor touches one file.

## VERIFICATION

- **232 tests across 17 files**, including an in-memory PostgREST double that enforces RLS the way Postgres does.
- Covered: cross-user isolation, memory, all tools, cooking state, barge-in, filler timing, state recovery after an interruption, and latency budgets.
- A preflight command checks every live provider — including that the *fallback* model is actually served by the account, a failure that is otherwise invisible until the worst moment.
- Deployed on Vercel; anonymous callers are refused by every API route in production.